

Lista 02 - Machine Learning IMPA

Pedro Barbosa Bahia

20 de fevereiro de 2026

Conteúdo

Exercício 1	3
Item a	3
Item b	4
Item c	5
Item d	6
Item e	7
Item f	8
Item g	9

Exercício 1

Item a

A partir da função de custo para Ridge Regression:

$$L = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Derivando em relação a β_0 e igualando a zero, temos:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n 2 \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right) (-1) = 0$$

$$-2 \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right) = 0$$

$$\sum_{i=1}^n y_i - n\beta_0 - \sum_{i=1}^n \sum_{j=1}^p x_{ij} \beta_j = 0$$

Isolando β_0 :

$$\beta_0 = \frac{1}{n} \sum_{i=1}^n y_i - \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^p x_{ij} \beta_j$$

$$\beta_0 = \bar{y} - \sum_{j=1}^p \bar{x}_j \beta_j$$

Caso os dados estejam centralizados, teremos $\bar{y} = 0$ e $\bar{x}_j = 0$. Portanto, $\hat{\beta}_0 = 0$.

Item b

A função de custo na forma matricial, com $\beta_0 = 0$, é:

$$L = (Y - X\beta)^T(Y - X\beta) + \lambda\beta^T\beta$$

Derivando em relação ao vetor β e igualando a zero, temos:

$$\frac{\partial L}{\partial \beta} = -2X^T(Y - X\beta) + 2\lambda\beta = 0$$

$$-X^TY + X^TX\beta + \lambda\beta = 0$$

$$(X^TX + \lambda I)\beta = X^TY$$

O estimador em forma fechada é:

$$\hat{\beta} = (X^TX + \lambda I)^{-1}X^TY$$

Assumindo que $\lambda \rightarrow 0$, o termo de regularização desaparece e temos:

$$\hat{\beta} = (X^TX)^{-1}X^TY$$

que é a fórmula fechada clássica para a Regressão Linear sem regularização.

Item c

A partir do estimador em forma fechada encontrado no item (b):

$$\begin{aligned}\hat{\beta} &= (X^T X + \lambda I)^{-1} X^T Y \\ &= (\lambda I + X^T I X)^{-1} X^T Y \\ &= [(\lambda I)^{-1} - (\lambda I)^{-1} X^T (I + X(\lambda I)^{-1} X^T)^{-1} X(\lambda I)^{-1}] X^T Y \\ &= \left[\frac{1}{\lambda} I - \frac{1}{\lambda} X^T \left(I + \frac{1}{\lambda} X X^T \right)^{-1} X \frac{1}{\lambda} \right] X^T Y \\ &= \frac{1}{\lambda} \left[X^T - X^T \left(I + \frac{1}{\lambda} X X^T \right)^{-1} \left(\frac{1}{\lambda} X X^T \right) \right] Y \\ &= \frac{1}{\lambda} X^T \left[I - \left(I + \frac{1}{\lambda} X X^T \right)^{-1} \left(\frac{1}{\lambda} X X^T \right) \right] Y \\ &= \frac{1}{\lambda} X^T \left[I - \left(I + \frac{1}{\lambda} X X^T \right)^{-1} \left(I + \frac{1}{\lambda} X X^T - I \right) \right] Y \\ &= \frac{1}{\lambda} X^T \left[I - I + \left(I + \frac{1}{\lambda} X X^T \right)^{-1} \right] Y \\ &= X^T \left[\frac{1}{\lambda} \left(I + \frac{1}{\lambda} X X^T \right)^{-1} Y \right]\end{aligned}$$

Portanto, podemos escrever $\hat{\beta} = X^T \alpha$, onde o vetor α é dado por:

$$\begin{aligned}\alpha &= \frac{1}{\lambda} \left(I + \frac{1}{\lambda} X X^T \right)^{-1} Y \\ \alpha &= (\lambda I + X X^T)^{-1} Y\end{aligned}$$

Item d

Para um novo ponto x^* , a previsão $\hat{y}^*$ é:

$$\hat{y}^* = (x^*)^T \hat{\beta}$$

Substituindo $\hat{\beta} = X^T \alpha$ encontrado no item (c):

$$\hat{y}^* = (x^*)^T X^T \alpha$$

$$\hat{y}^* = (x^*)^T X^T (\lambda I + X X^T)^{-1} Y$$

Item e

Seja $K(x, z) = \langle \phi(x), \phi(z) \rangle$ uma função de kernel associada a um mapeamento $\phi(\cdot)$ para um espaço de características.

No item anterior, a predição foi escrita como

$$\hat{y}^* = (x^*)^T X^T \alpha$$

Definindo o vetor de kernels

$$K(x^*, X) = \begin{bmatrix} K(x^*, x_1) \\ K(x^*, x_2) \\ \vdots \\ K(x^*, x_n) \end{bmatrix},$$

podemos escrever a predição em forma matricial como

$$\hat{y}^* = K(x^*, X)^T \alpha.$$

Item f

No cálculo da função de custo e na otimização do SVM, apenas os vetores de suporte são considerados. Pelas condições de KKT, temos que $\alpha_i > 0$ apenas para os pontos onde a restrição de margem é ativa, ou seja, em pontos dentro da margem ou incorretamente classificados (os vetores de suporte). Para todas as outras amostras, $\alpha_i = 0$. Isso faz com que o vetor α no SVM seja altamente esparsos.

Enquanto isso, na Ridge Regression, os valores de α_i calculados raramente serão nulos. De modo que a quase totalidade das amostras de treino contribuirá para a soma final da previsão $\hat{y}^*$. Devido à esparsidade do vetor α no SVM, ele se torna computacionalmente mais eficiente.

Item g

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split, KFold
4 from sklearn.linear_model import Ridge, LinearRegression
5 from sklearn.kernel_ridge import KernelRidge
6 from sklearn.pipeline import make_pipeline
7 from sklearn.preprocessing import StandardScaler
8 from tqdm import tqdm
9 from sklearn.metrics import mean_squared_error
10
11 number_of_folds = 10
12 data = pd.read_csv("../data/bodyfat.csv")
13 X = data.drop(columns=["BodyFat", "Density"])
14 y = data["BodyFat"]
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, random_state=0
17 )
```

(i) Erro médio quadrático da regressão linear

```
1 linear_regression_model = LinearRegression(fit_intercept=True)
2 linear_regression_model.fit(X_train, y_train)
3 y_hat = linear_regression_model.predict(X_test)
4 mse_linear = mean_squared_error(y_true=y_test, y_pred=y_hat)
5
6 y_hat_train = linear_regression_model.predict(X_train)
7 mse_linear_train = mean_squared_error(y_true=y_train, y_pred=
  y_hat_train)
```

Conjunto	Erro médio quadrático
Teste	15.910489104854445
Treino	18.090133641655044

Tabela 1: Resultados da regressão linear

(ii) Erro médio quadrático da regressão ridge

```
1 alphas = 10 ** np.linspace(3, -2, 100)
2 ridge_cv_mse = []
3 for alpha in tqdm(alphas):
4     cv_MSE = []
5     folds = KFold(
6         n_splits=number_of_folds, shuffle=True, random_state=2023
7     ).split(X_train, y_train)
8
9     for train_idx, val_idx in folds:
10         ridge_pipeline = make_pipeline(
11             StandardScaler(), Ridge(alpha=alpha)
12         )
13         ridge_pipeline.fit(
14             X_train.iloc[train_idx], y_train.iloc[train_idx]
15         )
16         y_hat = ridge_pipeline.predict(
17             X_train.iloc[val_idx]
18         )
19         cv_MSE.append(
20             mean_squared_error(y_true=y_train.iloc[val_idx], y_pred=
21                 y_hat)
22         )
23
24     ridge_cv_mse.append(np.mean(cv_MSE))
25
26 optimal_alpha = alphas[np.argmin(ridge_cv_mse)]
27 ridge_pipeline = make_pipeline(
28     StandardScaler(), Ridge(alpha=optimal_alpha)
29 )
30 ridge_pipeline.fit(X_train, y_train)
31 y_hat_ridge = ridge_pipeline.predict(X_test)
32 ridge_test_mse = mean_squared_error(y_true=y_test, y_pred=
33     y_hat_ridge)
34
35 y_hat_ridge_train = ridge_pipeline.predict(X_train)
36 ridge_train_mse = mean_squared_error(
37     y_true=y_train, y_pred=y_hat_ridge_train
38 )
```

O melhor valor de α encontrado foi **0.52**. Com ele, obteve-se os seguintes erros médios quadráticos:

Conjunto	Erro médio quadrático
Validação (10 folds)	21.84
Teste	15.87
Treino	18.099

Tabela 2: Resultados da regressão ridge

(iii) Kernel para Ridge Regression

```
1 kernels = ["linear", "polynomial", "rbf", "laplacian"]
2 gammas = [10**-3]
3 alphas = 10 ** np.linspace(3, -2, 100)
4 hyperparams = [
5     (kernel, gamma, alpha)
6     for kernel in kernels
7     for gamma in gammas
8     for alpha in alphas
9 ]
10
11 kernel_ridge_cv_mse = []
12 for hyperparam in tqdm(hyperparams):
13     kernel_fold, gamma_fold, alpha_fold = hyperparam
14     cv_MSE = []
15     folds = KFold(
16         n_splits=number_of_folds, shuffle=True, random_state=2023
17     ).split(X_train, y_train)
18
19     for train_idx, val_idx in folds:
20         kernel_ridge_pipeline = make_pipeline(
21             StandardScaler(),
22             KernelRidge(
23                 kernel=kernel_fold,
24                 alpha=alpha_fold,
25                 gamma=gamma_fold
26             ),
27         )
28         kernel_ridge_pipeline.fit(
29             X_train.iloc[train_idx], y_train.iloc[train_idx]
30         )
31         y_hat = kernel_ridge_pipeline.predict(X_train.iloc[val_idx])
32
33         cv_MSE.append(
34             mean_squared_error(y_true=y_train.iloc[val_idx], y_pred
35                               =y_hat)
36         )
37
38         kernel_ridge_cv_mse.append(np.mean(cv_MSE))
39
40 optimal_hyperparam = hyperparams[np.argmin(kernel_ridge_cv_mse)]
41 optimal_kernel, optimal_gamma, optimal_alpha = optimal_hyperparam
42
43 kernel_ridge_pipeline = make_pipeline(
44     StandardScaler(),
45     KernelRidge(
46         kernel=optimal_kernel, alpha=optimal_alpha, gamma=
47         optimal_gamma
48     ),
49 )
50 kernel_ridge_pipeline.fit(X_train, y_train)
51 kernel_y_hat_ridge = kernel_ridge_pipeline.predict(X_test)
52 kernel_ridge_test_mse = mean_squared_error(
53     y_true=y_test, y_pred=kernel_y_hat_ridge
54 )
```

```

53 kernel_y_hat_ridge_train = kernel_ridge_pipeline.predict(X_train)
54 kernel_ridge_train_mse = mean_squared_error(
55     y_true=y_train, y_pred=kernel_y_hat_ridge_train
56 )

```

O melhor conjunto de hiperparâmetros encontrado é: este é: 14.66379896222263
 reino é: 17.223450675066484

Hiperparâmetro	Valor
α	0.01
γ	0.001
kernel	rbf

Com ele, os erros médios encontrados foram:

Conjunto	Erro médio quadrático
Validação (10 folds)	20.55
Teste	14.66
Treino	17.22

Tabela 3: Resultados da regressão ridge com kernel

(iv) Comparação dos modelos

Resultados no conjunto de treino:

Modelo	Erro médio quadrático
Regressão linear	18.09
Regressão ridge	18.53
Regressão ridge com kernel	17.22

Tabela 4: Resultados de treino

Resultados no conjunto de teste:

Modelo	Erro médio quadrático
Regressão linear	15.91
Regressão ridge	15.87
Regressão ridge com kernel	14.66

Tabela 5: Resultados de teste

Dos três modelos analisados, o que apresentou melhor desempenho foi a regressão ridge com kernel, tanto no conjunto de treino quanto no conjunto de teste. A regressão linear e a regressão ridge apresentaram desempenhos semelhantes, com a regressão ridge tendo um desempenho ligeiramente pior no conjunto de treino, mas melhor no conjunto de teste. Isso sugere que a regressão ridge pode ter ajudado a reduzir o overfitting em comparação com a regressão linear, enquanto a adição do kernel na regressão ridge permitiu capturar relações não lineares nos dados, resultando em um desempenho superior.